

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Systems and Methods for Big and Unstructured Data Project

Author(s): **Francesco Riccardi**
Alessandro Romito
Marco Luca Previtera
Simone Lo Gatto
Lorenzo Ferretti

Group Number: **9**

Academic Year: 2022-2023

Contents

Contents **i**

1 Introduction **1**
1.1 Problem Specification 1
1.2 Hypothesis 1

2 ER Diagram **3**
2.1 Entities and Relationships 3
2.2 ER Diagram description 3

3 Graph Diagram **5**
3.1 Graph Diagram description 5
3.2 ER Diagram and Neo4j difference 7

4 Dataset Description and processing **9**

5 Queries and Commands **15**
5.1 Journal's authors 15
5.2 Author's work percentage 16
5.3 Published articles per period 17
5.4 Published work per period 18
5.5 19
5.6 Extra 20
5.6.1 Import in the Database 20
5.6.2 Creation Coauthor relationship 21
5.6.3 Creation Cite relationship 22

6 Conclusions **25**

A Appendix A **27**

List of Figures **29**

List of Tables **31**

1 | Introduction

1.1. Problem Specification

The aim of this project was to design a 'query graph data structure' in Neo4j for supporting a bibliographic Database. The database must store a large scale Dataset recording publications with their features and their descriptions in order to have an overview of the library.

1.2. Hypothesis

The assumptions taken into account are the following:

- All data are assumed to be truthful without further verification;
- We are using the concept of publication referring to Book, Article, Thesis, Online Resource, Proceeding, Inproceeding and Incollection, instead, Journal isn't belonging to this family of Entities;
- Whoever uploads the data relating to publications does so by entering all the information in a correct and complete way;
- An author can't publish the same work more than once so that redundancy and duplicates are avoided in the system;
- Every publication has an associated dynamic attribute "Score" that computes the number of works that quote a specific one. As said, this attribute is dynamic and any change due to references updating can be managed through different approaches:
    1. The attribute can be kept inconsistent in the db until the next reading when it will be recalculated. This is the worst case scenario because the number of reading is much higher than the writing one,therefore this behavior is inefficient;
    2. The attribute can be kept inconsistent in the db until it will be recalculated by a trigger after a certain amount of time. This context is not entirely problem-

free because there are small chances that the trigger doesn't update the value in time.

3. The attribute is always kept consistent after an applied operation. All this is fine because the writing operation frequency is lower than the reading one.

- Cite is a randomly generated relationship between publications, the references don't quote works outside the local db.
- The editor of the publication is consider as the author.

2 | ER Diagram

2.1. Entities and Relationships

The designed ER diagram contains the following entities: Person generic, Researcher, Book, Article, Journal, Proceeding, Inproceeding, Incollection, Publisher, Thesis, Online Resource. Some classes have been introduced due to ER diagram correctness reasons, but they haven't been taken into account during the design of the Graph Database for ease of management of the DB. Entities are retrieved through a process of data analysis in which the hierarchy has been structured according to the XML file, instead, relationships and attributes are selected in a way that only the most relevant ones are kept for the project.

2.2. ER Diagram description

The Person Generic Entity is a Superclass identified by Name and Surname and its specialized subclass is Researcher identified by Orcid which provides a persistent digital identifier distinguishing every researcher, this choice has been made to differentiate an amateur that doesn't make any formal publication from a professional researcher. The generalization is disjoint and partial. Every publication such as Book, Article, Thesis, Online Resource, Proceeding, Inproceeding, Incollection is made unique by its key and its publication date. Proceeding is a collection of academic papers published in the context of an academic conference or workshop which contains the contributions made by researchers(Inpreceeding) at the conference. The Entity Incollection is referred to the entries that are written from an author that is different from the one that has drawn up and edited the book. Volume, Number and Series, as said before, have been introduced due to correctness aspect but they are represented only as relationship in the graph diagram. Every Journal has its name, Volume is identified by Journal and Volume Number, Number is identified by Volume and Number. Series are collections of publications which refer to a common topic or theme, they are identified by their title. These are the attributes contained by each relationship:

- Book_in between Book and Incollection: Page end and Page start;

- incollection_Series: Number;
- Book_Series: Number;
- Preceeding_Series: Number;
- Article_Number: Page end and Page start.

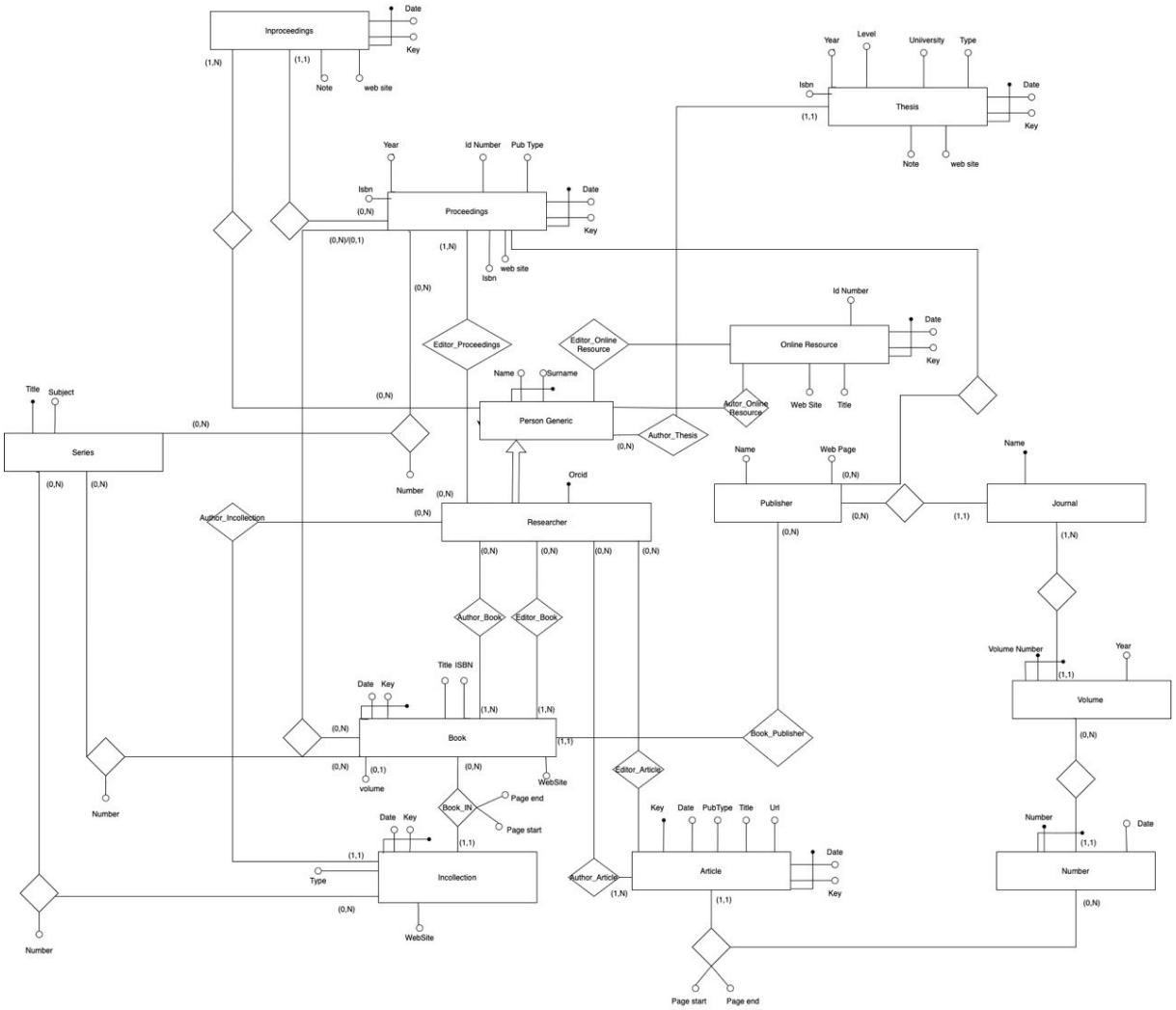


Figure 2.1: ER Diagram

3 | Graph Diagram

3.1. Graph Diagram description

Author, Article, Journal, Book, Incollection, Proceedings, Inproceedings, Www, Mastersthesis, University and Series are represented as nodes and their attributes are shown in the graph diagram below. The relationships of the database are WRITTEN_BY, CONTAIN, PUBLICATED_BY, COAUTHOR, AT, CITE, INCLUDE. The nodes Article, Book, Incollection, Proceedings, Inproceedings, Www, Mastersthesis contain the Publication tag in order to perform and understand queries easily since you don't need to specify 7 different nodes in the MATCH statement every time. The relationships between nodes are the following:

```
(Pubblication)-[WRITTEN_BY]->(Author)
(Proceedings)-[CONTAIN]->(Inproceedings)
(Book)-[CONTAIN]->(Incollection)
(Mastersthesis)-[AT]->(University)
(Author)-[PAST_STUDENT]->(University)
(a1:Author)-[COAUTHOR]-(a2:Author)
(p1:Publication)-[CITE]->(p2:Publication)
(Article)-[PUBLICATED_BY]->(Journal)
(Serie)-[INCLUDE]->(Publication)
```

Publication and Author nodes have the citationScore attribute that is used to store a computed index which describes how many times that node is quoted. This attribute is updated by using a specific command that executes the PageRank algorithm on the graph. The citationScore attribute will be used in queries to create ranks. The relationship COAUTHOR is introduced to increase the speed of the queries searching for the relationships between authors. ~~They are computed using a specific command after the data is imported in the DB.~~ Two authors are connected by the coauthor relationship only if they have participated to the writing of the same publication. The implementation of the relationship PAST_STUDENT was made to optimize the queries on the MasterThe-

sis publication type. Its purpose is to associate an author who wrote a thesis directly with its university. Journals are represented as nodes and described only by their name, it has a relationship with Article sharing the publication details (year, month, volume, pages, number). By doing this it is possible to efficiently aggregate all the articles published in the same journal without creating one node for each number of that journal. The database design decision follows two thoughts:

1. be fast, the core of the structure is able to easily guarantee the functioning of complex algorithm like shortest path, thanks to the quantity of relations;
2. be used for ranking authors and articles based on citations. ~~We developed citationScore, based on pageRank algorithm that allows to create different formulas to analyze data in the most appropriate way.~~

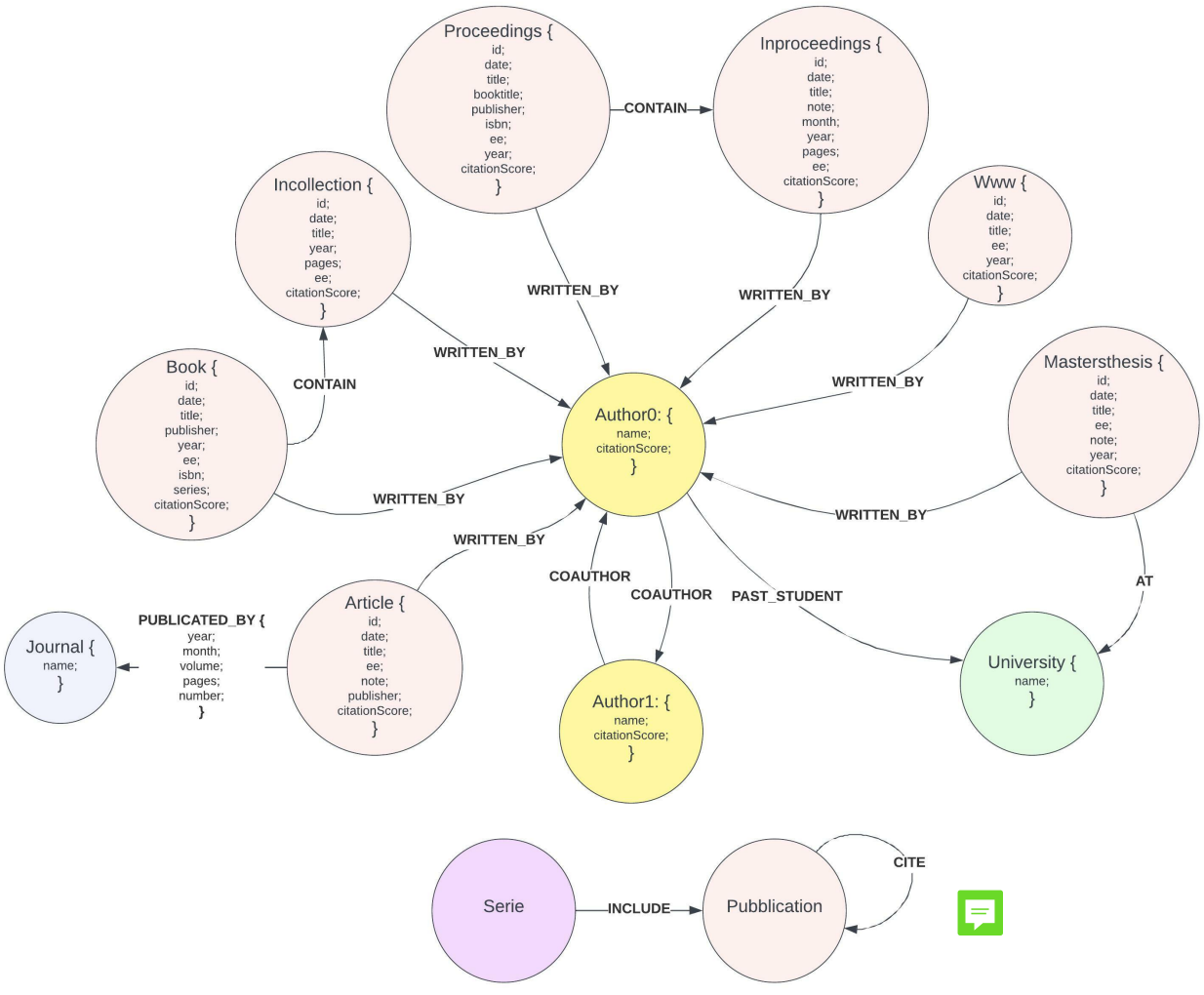


Figure 3.1: Graph Diagram

3.2. ER Diagram and Neo4j difference

The complete and general ER model that has been created represents the real world of scientific bibliography.

As requested in neo4j we worked in order to make the most from the tool used and the type of database.(Non capisco)

There are some differences in the treatment of the authors in neo4j (simple nodes with some attributes) and the authors in the ER diagram (researchers + generalization in a generic person) to differentiate the ones with ORCID and the ones without, since in reality only those who own the ORCID can write a book or scientific article, while students or other generic people can still speak in conferences (Proceedings), write online resources (Www) or write a thesis (Mastersthesis).Journal entity is more detailed in the ER diagram (with weak entities such as volume and number reflecting several more specific releases, and an entity “Publisher” that publish a book or a proceeding on the journal) then the graph diagram with some attributes in the relationship “Publicated_by”. The University node is not an entity in the ER diagram, but only a simple attribute, that’s because there are a few Universities connected only for thesis, the node is a way to create a circular relationship with the author (membership relationship to a school that is not explicit in the ER diagram to avoid redundancy). We have several authors in the graph diagram to have a "co-author" relationship that simply are authors of the same publications (or other). Finally, the difference in the treatment of the series entity which, thanks to the parent entity "Publication", is treated in a more orderly and concise way in the ER diagram compared to neo4j (due to the absence of true generalizations).

4 | Dataset Description and processing

The chosen Dataset is the DBLP computer science bibliography. It is the on-line reference for bibliographic information on major computer science publications. As January 2019 DBLP contains over 4.4 million publications, published by more than 2.2 million authors. The Dataset is divided in different types of publication (article, book, incollection, proceedings, inproceedings, www, mastersthesis) followed by their features and details. The attributes and relationships of these types are already described in the ER Diagram section 2. The Dataset has been subject to a process of elaboration and extraction in order to clear the data and take care of their quality, indeed, limited quality data could "cause a series of consequences for efficiency". The starting point was cut the 3.5 Gigabyte of data into 100 Megabyte (more or less 200.000 nodes) and convert its XML format into CSV. These operations were made much easier through the use of two Python programs:

1. The first one 4.1 aims at converting the entity names or entity numbers of the reserved HTML characters with its original character representation the encoded UTF-8 string and at cleaning up the non-standard xml code. It is important to highlight that the program realizes the reading of the first 100 Megabyte of the Dataset performing a data cut.

```
import html
import fileinput

totMB = 100

inputFile="D:/Uni/Magistrale - 1/SMBUD/Progetto/Data/dblp.xml"
outputFile="D:/Uni/Magistrale - 1/SMBUD/Progetto/Data/out"+str(totMB)+"M.xml"

input = open(inputFile,'rb')
text = input.read(totMB*1000000)
output = open("temp.xml",'wb')
output.write(text)
output.close()

output = open(outputFile, 'a', encoding="UTF-8")
output.truncate(0)
count = 0

for lines in fileinput.input(["temp.xml"]):
    line = lines.replace("<i>", "")
    line = line.replace("</i>", "")
    line = line.replace("<sup>", "")
    line = line.replace("</sup>", "")
    line = line.replace("<sub>", "")
    line = line.replace("</sub>", "")
    line = line.replace("&lt;", "")
    line = line.replace("&gt;", "")
    line = line.replace('&amp;', " and ")
    uLine = html.unescape(line)
    output.write(uLine)
    count+=1
    if count % 1000000 == 0:
        print(count)


output.close()
```

Figure 4.1: First Python program

2. The second one 4.2 carries out the conversion from xml to csv and removes the special characters from csv. One more thing to mention is that after all types of publications are found only a limited number is stored for each type (e.g. a maximum number of the type book and of the type article is saved fixing an upper bound at 2000).

```python
import xml.etree.ElementTree as ET
import csv
import math
import random
import copy

input_file = "out100M.xml"
num_limit = 2000

tree = ET.parse(input_file)
root = tree.getroot()

csvIndex = {}
csvHeaders = {}
keyListProceedings = []
keyListBooks = []

#find all types of pubblications and loads to a dictionary
for child in root:
    if child.tag not in csvIndex:
        csvIndex[child.tag]={}
        csvHeaders[child.tag]=[]

#scan all pub types
#maxDict object to store the maximum number of attributes for each pubblication type
for pub_type in csvIndex.keys():
    maxDict = {}
    currIndex = 2
    counter = 0

    for elem in root.findall(pub_type):
        thisDict = {}

        if pub_type == "proceedings":
            keyListProceedings.append(elem.attrib["key"])
            #save the key of that article in a list for randomic assigment

        if pub_type == "book":
            keyListBooks.append(elem.attrib["key"])
            #save the key of that article in a list for randomic assigment

        for innerTag in elem:
            #counts the number of each tags for an element
            if innerTag.tag in thisDict:
                thisDict[innerTag.tag]+=1
            else:
                thisDict[innerTag.tag]=1
```

```
48                #we create the tag in the dictionary
49
50            #updates the total count for that pubblication type
51            if innerTag.tag in maxDict:
52                if thisDict[innerTag.tag] > maxDict[innerTag.tag]:
53                    maxDict[innerTag.tag] = thisDict[innerTag.tag]
54            else:
55                maxDict[innerTag.tag]=1
56                #we create the tag in the dictionary
57
58        counter += 1
59        if counter>=num_limit:
60            break
61
62    #load the found number of attributes for
63    #that pubblication type in the global model of indexes
64    for key in maxDict.keys():
65        csvIndex[pub_type][key] = currIndex
66        currIndex = currIndex+maxDict[key]
67
68        csvHeaders[pub_type] = ["date","key"]
69        for key in maxDict.keys():
70            #creates the header for the csv
71            for n in range(maxDict[key]):
72                csvHeaders[pub_type].append(key+str(n))
73
74
75
76 #loads the data in the csv one type at time
77 for pub_type in csvIndex.keys():
78     counter = 0
79     out_csv = open("./csv/"+pub_type+'.csv','w', newline='', encoding='UTF-8')
80     csvwriter = csv.writer(out_csv)
81
82     csvwriter.writerow(csvHeaders[pub_type])
83     #write the header row
84
85     for elem in root.findall(pub_type):
86         row = [""] * len(csvHeaders[pub_type]) #array initialization
87         tmpIndex = copy.deepcopy(csvIndex)
88         #duplicate the index model for writing
89
90         #load fixed attributes of pubblications
91         row[0] = elem.attrib["mdate"]
92         row[1] = elem.attrib["key"]
93
94         #load dynamic values of tags
```

```
95          for innerTag in elem:
96              row_pos = tmpIndex[pub_type][innerTag.tag] #get the write column position
97              if innerTag.tag == "crossref" and pub_type == "incollection":
98                  row[row_pos] = random.choice(keyListBooks)
99              elif innerTag.tag == "crossref" and pub_type == "inproceedings":
100                 row[row_pos] = random.choice(keyListProceedings)
101             else:
102                 row[row_pos] = innerTag.text
103                 #write the corrisponding value
104
105             tmpIndex[pub_type][innerTag.tag]+=1
106             #increment the writing position
107
108         csvwriter.writerow(row)
109         counter += 1
110         if counter>=num_limit:
111             break
112
113     out_csv.close()
```

Figure 4.2: Second Python program

The final csv result of this process is further analyzed and elaborated by deciding what to include or not in the Neo4j DB based on quantity of data present (erasing the weak and irrelevant data and relations). After doing this, the relationship between the field Book_Incollection and Proceeding_Inproceeding are generated randomly because the Database is not considered in its entirety, indeed, it is not certain that the original correlations have been remained unaltered or have been lost during the cutting operation. The last step has been the DB upload in Neo4j through the cypher queries in particular using the command Load as we will see in Query Chapter.

5 | Queries and Commands

5.1. Journal's authors

This query finds for the specify journal X the authors that have written for it and the number of their respective publications in a certain period of time. The matched results are sorted in descendent through the Order by clause and the query returns a limited subset of the rows through the Limit clause. In this example the query returns the top 10 author of IWBS Report during the period between 1990 to 2000.
Complexity 3 nodes + conditions + aggregations + limit

```
WITH '1990' AS startYear, '2000' AS endYear, 'IWBS Report' AS journalName
MATCH (aut:Author)-[:WRITTEN\_BY]-(a:Article)-[r:PUBLICATED\_BY] ->(j:Journal)
WHERE j.name = journalName AND r.year >= startYear AND r.year <= endYear
WITH aut, j, COUNT(*) AS numArtXAut
ORDER BY numArtXAut DESC
RETURN aut.name, numArtXAut
LIMIT 10
```

	Name of the author	Number of articles x author
1	Christoph Beierle	8
2	Egon Börger	7
3	Stefan Böttcher	6
4	Ulrike Schwall	5
5	Stefan J. Schierholz	4
6	Udo Pletat	4
7	Gregor Meyer	3
8	Jochen Dörre	3
9	Ulrike Weiland	3
10	Claudia Maienborn	3

5.2. Author's work percentage

This query returns for each work of a specific author the number of entries(Incollection) that he has written in that Book, this values is computed through the ratio between the number of entries of the book written by the author and the total number of entries of the book. At the end we use the Case clause to generate a conditional expression that returns the book's name and 0 if the author didn't write any entries of the book or the book doesn't have any entries. In this case the query returns the percentage of entries for each book written by Tor Helleseth.
Complexity 3 nodes + conditions + aggregations

```
MATCH (inc:Incollection)-[:WRITTEN_BY]->(aut:Author),(b:Book)-[:CONTAIN]->(inc)
WHERE aut.name = 'Tor Helleseth'
WITH b, COUNT(*) AS numIncXBook
MATCH (b)-[:CONTAIN]-(inc:Incollection)
WITH b, numIncXBook, COUNT(*) AS numTotIncXBook
RETURN b.title,
CASE
WHEN numIncXBook = 0 THEN 0.0
WHEN numTotIncXBook = 0 THEN 0.0
ELSE (numIncXBook * 1.0 / numTotIncXBook * 1.0) * 100.0 END AS Percentage
```

	Book title	Rating
1	Handbook on Graph Drawing and Visualization	5.263157894736842
2	Recommender Systems Handbook	4.166666666666666
3	Encyclopedia of Complexity and Systems Science	11.76470588235294
4	Handbook of Discrete and Computational Geometry, Second Edition	12.5
5	"Encyclopedia of Parallel Computing"	5.2631578947368425
6	Encyclopedia of Data Warehousing and Mining (4 Volumes) Second Edition (4 Volumes)	4.545454545454546
7	Information security management handbook, Sixth Edition	5.555555555555555
8	Encyclopedia of Computational Neuroscience	5.0
9	Handbook of Algorithms for Physical Design Automation	4.3478260869565215
10	Handbook of Formal Languages, Volume 2. Linear Modeling: Background and Application	3.8461538461538463
11	Handbook of Information and Communication Security	4.545454545454546
12	Handbook of Semantic Web Technologies	6.25
13	Handbook of Mathematical Methods in Imaging	6.666666666666667
14	Handbook of Virtual Environments - Design, Implementation, and Applications, Second Edition	5.0
15	Handbook of Spatial Logics	5.263157894736842
16	Handbook of Database Security - Applications and Trends	4.3478260869565215
17	The Handbook of Mobile Middleware	6.666666666666667

5.3. Published articles per period

This query returns the article of journal with their respective author that are the published in a certain period of time. The result is sorted in descendent by the publication date and it is a limited subset of the rows through the Limit clause. In this case the query returns in descendent order the 10 article of journal with their respective author during the period between 1990 to 2000.
Complexity 3 nodes + conditions + aggregations+limit

```
MATCH (aut:Author)<-[:WRITTEN_BY]-(a:Article)-[r:PUBLICATED_BY]->(j:Journal)
WHERE r.year>='1980' and r.year<= '1990'
WITH a.title AS articleTitle,r.year AS pubYear,aut.name as Autore
ORDER BY pubYear DESC
RETURN articleTitle,Autore, pubYear
LIMIT 10
```

	articleTitle	author	pubblication year
1	Object Data Model Facilities for Multimedia Data Types	Mark F. Hornick	1990
2	Object Data Model Facilities for Multimedia Data Types	Frank Manola	1990
3	Object Data Model Facilities for Multimedia Data Types	Alejandro P. Buchmann	1990
4	Object Oriented DBMS as Generalization of Relational DBMS	David Beech	1990
5	Object Oriented DBMS as Generalization of Relational DBMS	Cetin Ozbutun	1990
6	Modification of User Defined Types	Phil Shaw	1990
7	Modelling Spatial Knowledge on a Linguistic Basis	Geoffrey Simmons	1990
8	Modelling Spatial Knowledge on a Linguistic Basis	Ewald Lang	1990
9	Modelling Spatial Knowledge on a Linguistic Basis	Kai-Uwe Carstensen	1990
10	Das Problem der VP-Ellipsen im Rahmen einer Theorie der Sachverhaltsanaphern	Elke Busch	1990 1990

5.4. Published work per period

This query returns the title of works published by a specific journal over the last year with the authors who wrote it. The max() function returns the maximum value in a set of values and after the matching condition are applied. In this case the query returns the works published in the last year by the journal IWBSReport and their authors
Complexity 3 nodes + conditions + aggregations+limit

```
MATCH(a:Author)<-[:WRITTEN_BY]-(p:Publication)-[r:PUBLICATED_BY]-(j:Journal)
```

```
WITH j,r,p,a,max(r.year) AS maxYear
WHERE j.name = 'IWBSReport' AND r.year=maxYear
RETURN p.title,collect(a.name) AS authors
LIMIT 10
```

	p.title	authors
1	ILFA Interface Concept	["Wolfgang Schönfeld", "Theodor Lettmann", "Bernd I. Dahn"]
2	Modelling Spatial Knowledge on a Linguistic Basis	["Geoffrey Simmons", "Ewald Lang", "Kai-Uwe Carstensen"]
3	Domain Restriction and Other Kinds of Reference Set Operations in Sentence and Discourse Semantics.	["Rolf Mayer"]
4	Das Problem der VP-Ellipsen im Rahmen einer Theorie der Sachverhaltsanaphern	["Elke Busch"] ["Elke Busch"]
5	Syntaktische Informationseinheiten zu deutschen Adjektiven	["Stefan J. Schierholz"]
6	Complexity of Logical Decision Problems: An Introduction	["Egon Börger"]
7	Position und Bewegung: Zur Semantik lokaler Verben	["Claudia Maienborn"]
8	On Subsumption and Semiunifaction in Feature Algebras	["William C. Rounds", "Jochen Dörre"]
9	Derivationsmorphologie in einem Übersetzungssystem	["Anne Schiller"]
10	Knowledge Representation in LILOG	["Kai von Luck", "Udo Pletat"]

5.5.



Utilizzo di una formula creata appositamente per rankizzare gli autori di un certo giornale in modo equilibrato (né troppo né troppo poco)
Complexity 3 nodes + conditions + aggregations+limit

```
MATCH (a:Author)<-[:WRITTEN_BY]-(p:Publication)-[:PUBLICATED_BY]->(j:Journal)
WHERE j.name='Sci. Eng. Ethics'
WITH collect(p) AS pubs, sum(p.citationScore) AS totScore,a, size(collect(p)) AS nu
RETURN a.name as Name, totScore, numPubs, totScore/numPubs AS meanScore, 0.1*totSco
ORDER BY FinalScore DESC
```

```
LIMIT 10
```

	Name	totScore	numPubs	meanScore	FinalScore
1	Michael D. Mumford	16.00781	16	1.00048	2.50122
2	Stephanie J. Bird	14.94547	15	0.99636	2.39127
3	Luciano Floridi	15.08879	16	0.94304	2.35762
4	Jaime A. Teixeira da Silva	13.24581	14	0.94612	2.17609
5	Neelke Doorn	13.00399	14	0.92885	2.13637
6	David B. Resnik	12.07406	12	1.00617	2.11296
7	Tanuj Kanchan	11.86092	12	0.98841	2.07566
8	Ana Marusic	11.64288	12	0.97024	2.03750
9	Shane Connelly	11.63754	12	0.96979	2.03657
10	Kewal Krishan	10.98351	11	0.99850	1.99700

query 7 non ha risultato perchè fa setting

5.6. Extra

This section introduces some of the necessary operation used to structure the db by creating Nodes and their relationships. These operations are essentials to perform the previous queries.

5.6.1. Import in the Database

As mentioned in the previous chapter, the DB is imported in Neo4j from csv file through the cypher queries in particular using the command Load. Below are shown a couple of example where Www, Authors and Editor are upload in the database with their attributes. The Merge clause ensures that a pattern exists in the graph when the specified pattern is not present and needs to be created, any variables previously bound to existing graph elements will be reused in the pattern. With Header clause we are specifying that the file starts with a single row containing column names allowing specific fields access by their corresponding column name.

```
#LOAD WWW
LOAD CSV WITH HEADERS FROM 'file:///www.csv' AS row
WITH row
MERGE (www:Www:Publication {id: row.key})
ON CREATE SET www.date = row['date'], www.title = row['title0'],
www.ee = row['ee0'], www.year = row['year0']
```

```
#LOAD AUTHORS
LOAD CSV WITH HEADERS FROM 'file:///www.csv' AS row
WITH row
MATCH (www:Www {id: row.key})
UNWIND range(0,4) AS i
WITH i, www, row WHERE row['author'+toString(i)] IS NOT NULL
MERGE (aut:Author {name: row['author'+toString(i)]})
MERGE (www)-[:WRITTEN_BY]->(aut)
```

```
#LOAD EDITORS AS AUTHORS
LOAD CSV WITH HEADERS FROM 'file:///www.csv' AS row
WITH row
MATCH (www:Www {id: row.key})
UNWIND range(0,5) AS i
WITH i, www, row WHERE row['editor'+toString(i)] IS NOT NULL
MERGE (aut:Author {name: row['editor'+toString(i)]})
MERGE (www)-[:WRITTEN_BY]->(aut)
```

5.6.2. Creation Coauthor relationship

This query creates a coauthor relationship between author of the same article and returns some data to analyze it. The list of author that has written the specified article is realized through the collect() function, then for each different authors the coauthor relationship is created, the query returns a limited subset of the rows through the limit clause.

```
MATCH (n:Article)-[w:WRITTEN_BY]->(a:Author)
WITH n,collect(a) AS authors
FOREACH( a1 IN authors |
FOREACH( a2 IN [a IN authors WHERE a<> a1| a]|
MERGE (a1)-[:COAUTHOR]-(a2)))
RETURN *
LIMIT 300
```

5.6.3. Creation Cite relationship

This query creates randomly Cite relationship between publication avoiding self-quotation. The range() returns a list comprising all integer values within a range bounded by a start value start and end value, in this case an interval included between 1 and 120 as citationRange is considered. The apoc.coll.randomItem(collection) function returns a random item from the list and it is used to randomly choose the number of cite that each publication has and which are the cite they are linked to,then for each different publication the Cite relationship is created.

```
WITH range(1,120) AS citationRange
MATCH(p:Publication)
WITH collect(p) AS publications, citationRange
MATCH(p2:Publication)
WITH p2, apoc.coll.randomItems(publications,
apoc.coll.randomItem(citatioRange))  AS citatedPublications
FOREACH (citated IN [c IN citatedPublications WHERE c <>p2 | c]  |
CREATE (p2)-[:CITE]->(citated))
```

```
WITH ["Spectre Attacks: Exploiting Speculative Execution.",
"An Evaluation of Object-Oriented DBMS Developments:
1994 Edition."] AS citatedList
CREATE(article:Article:Publication{citationScore:0.0,date:date(),
ee:"https://=)",id:" tr/marhanz/s00",title:"Happy Birthday Algoritm"})
MERGE(aut:Author {name:'Paul Kocher'})
CREATE (article)-[:WRITTEN_BY]->(aut)
MATCH(j:Journal)
WHERE j.name='meltdownattack.com'
CREATE (article)-[p:PUBLICATED_BY {year:date().year}]->(j)
MATCH(aCited1 :Article)
WHERE(aCited1.title="Spectre Attacks: Exploiting Speculative Execution.")
FOREACH[citatedpub IN citatedList |
CREATE (article)-[p:CITE]->(citatedpub),"update rank aCitated",
MATCH(citatedpub)-[:WRITTEN_BY]->(autList:Author) ]
```

```
FOREACH(aut IN autList| "update rank")
```

6 | Conclusions

In this chapter additional useful information are reported

A | Appendix A

If you need to include an appendix to support the research in your thesis, you can place it at the end of the manuscript. An appendix contains supplementary material (figures, tables, data, codes, mathematical proofs, surveys, ...) which supplement the main results contained in the previous chapters.

List of Figures

2.1 ER Diagram 4

3.1 Graph Diagram 6

4.1 First Python program 10
4.2 Second Python program 13

List of Tables